Managed abstractions

Resource	Program abstraction	Required enforcement
CPU and timer	Process or thread	Who runs, when control returns, who changes privilege
RAM and MMU	Address space	Ordinary programs cannot change others' mappings or permissions
Disk and controller	File and block service	Names, permissions, ordering, and error reporting
NIC	Socket and message	Endpoints, buffer ownership, packet and connection state

Convenience, utilization, response time, fairness, isolation, and reliability are separate goals. A long batch job can keep the CPU busy while interactive requests wait. Specify the workload and objective before claiming efficiency. The kernel is the trusted privileged core. Shells, GUIs, libraries, and daemons usually run in user space. A library call need not enter the kernel, and changing a shell need not change the kernel.

Policy and mechanism

Alice and Bob share a machine. Bob's program is untrusted. Policy denies Bob access to Alice's private pages and files and prevents permanent CPU monopolization. Mechanisms include protected page tables and memory-access checks, kernel file-authority checks, and a protected timer and scheduler entry.

An application's check is insufficient because Bob can bypass it. User and kernel modes are insufficient if the kernel reads sensitive files for Bob without validating authority. Enforcement must mediate every relevant operation; untrusted programs must not modify its policy or metadata. The boundary includes CPU control, memory, object rights, and mutable shared state. A hidden GUI button enforces nothing. Revoking an already authorized reference has a separate contract in Chapters 11 and 14.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Controller, driver, and DMA

The controller implements device control in hardware or firmware. The driver is OS software that understands its registers, queues, and completion status. CPU submission, device transfer, and CPU completion handling are distinct events. Polling or an interrupt can discover completion.

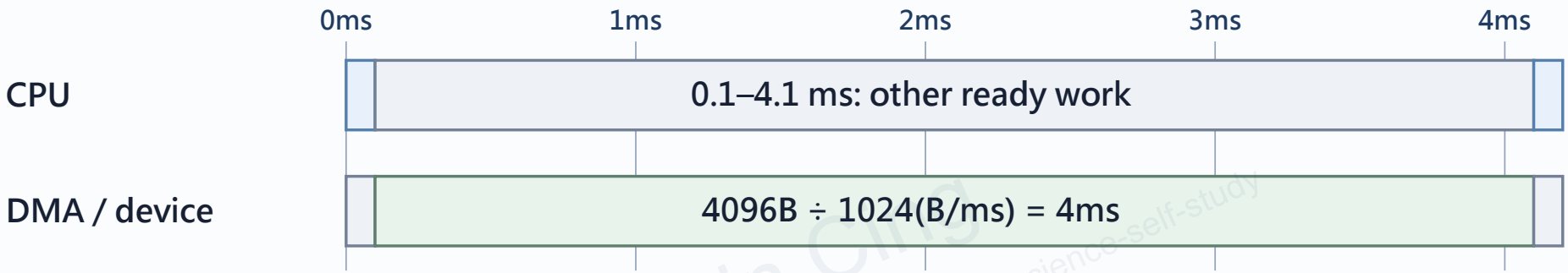
The fixed model gives setup 0–0.1 ms, DMA 0.1–4.1 ms, and completion handling 4.1–4.2 ms. During the middle 4 ms, the CPU can run another ready job. DMA reduces per-byte CPU copying but does not eliminate setup, cache synchronization, bandwidth, or completion costs. A 4096 B block at 1024 B/ms takes 4 ms. CPU control costs 0.2 ms, whereas request latency is 4.2 ms. CPU service time is not application waiting time. Required data visibility and DMA completion precede buffer return.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Controller, driver, and DMA (continued)

DMA: latency 4.2 ms; CPU control 0.2 ms

Blue: CPU control. Green: DMA. Gray: no transfer. Time is to scale.

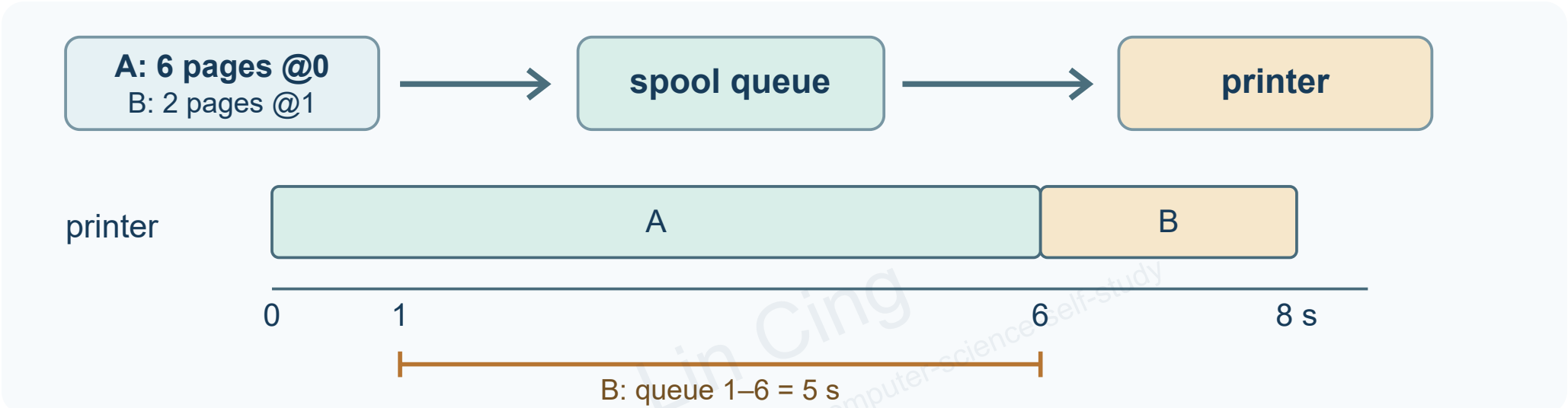


CPU setup 0–0.1 ms → DMA done 4.1 ms → CPU completion 4.1–4.2 ms

DMA avoids per-byte CPU copying; setup, ordering, completion, and buffer lifetime remain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Spooling: queued jobs and device completion



At one page per second with non-interleaved FCFS jobs, A's six pages arrive at t=0 and B's two at t=1. A prints during 0–6; B waits five seconds and prints during 6–8.

Submission confirms acceptance, not printing or survival after power failure. Jobs may reside on disk, in memory, or elsewhere; durability depends on the contract.

A spool and a buffer solve different problems

Mechanism	Managed object	Main purpose	Does not by itself imply
Spool / job queue	Complete jobs, order and state	Schedule a shared device; decouple submission from service	Durability, FCFS, or unlimited capacity
Buffer	Data temporarily held in transit	Absorb producer/consumer rate differences and batch transfers	RAM-only storage, completed work, or durability

They can be combined: the spool holds print jobs while the driver transfers each job in buffered chunks. When space runs out an API may block, reject, or report an error. Priority insertion also invalidates the FCFS timeline.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Interrupt, exception, and system call

Event	Relation to current instruction	Typical continuation
Device or timer interrupt	External, asynchronous	Save state, handle event, possibly schedule
Page fault or divide exception	Synchronous consequence	Repair and restart, report error, or terminate
System call	Deliberate synchronous request	Controlled entry, validation, service, return

ISA terminology and return points differ; not every exception resumes at the next instruction. An interrupt in kernel mode may cause no user-to-kernel transition, but still needs state preservation and reentry synchronization. An interrupt can return to the same thread, and a system call need not block. Neither implies a process switch. DMA can use polling or batched notification rather than one interrupt per block. State the completion rule.

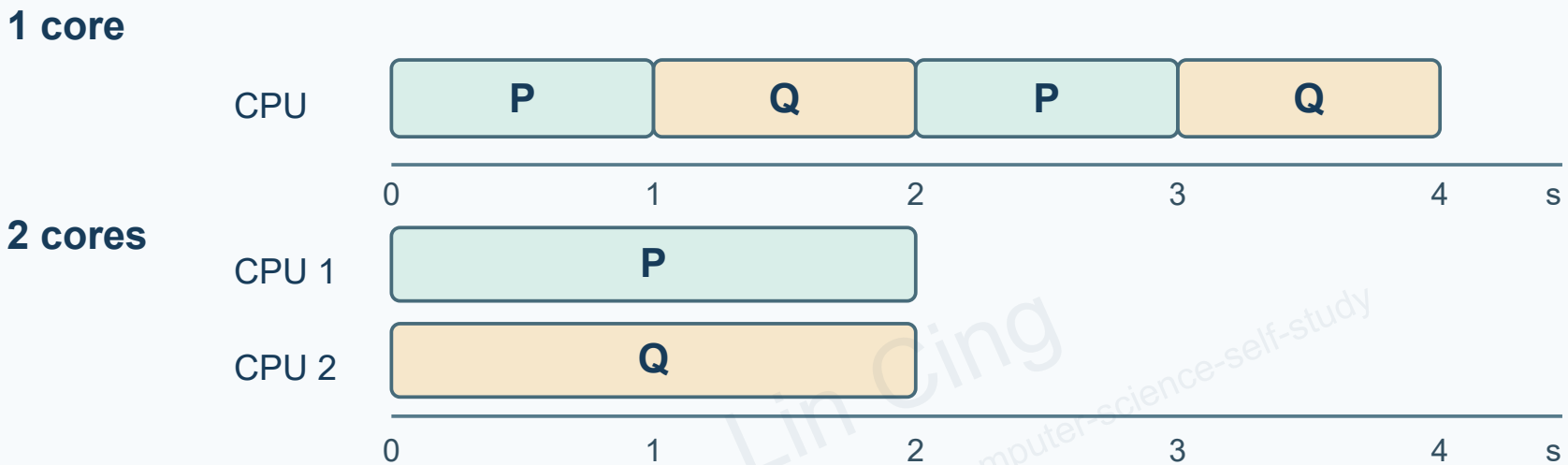
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Architecture follows sharing

Architecture	Typical sharing	Additional issue
One general-purpose CPU	One general execution resource	Concurrency with DMA and interrupts remains
SMP	Peer CPUs and common RAM	Synchronization and cache coherence
NUMA shared memory	Common addressable RAM, unequal locality	Location changes latency and bandwidth
Cluster	Independent networked nodes	Messages, partial failure, retry and consensus assumptions

Core, socket/package, and hardware thread count different things. SMP peer control and NUMA access asymmetry are separate dimensions; shared memory need not be UMA. Multiple CPUs alone do not establish fault tolerance. Power, RAM, kernel, or storage may fail jointly. Failover also needs detection, recovery, and exclusion of two simultaneous owners of an exclusive resource.

Concurrency and parallelism: read the timeline



Concurrency means tasks make progress in an overlapping period. On one core P and Q take turns.
Parallelism means work runs simultaneously on distinct resources, such as the two cores shown.

Shared data still needs synchronization. A single-core program can also run in parallel with DMA; state which resources are being compared.

NUMA mean and parallel speedup

Assume sequential nonoverlapping requests without cache effects. Local memory costs 100 ns, remote 180 ns. At 80% local access, mean cost is 0.8×100+0.2×180=116 ns. At 20% local it is 164 ns. Their ratio is not automatically whole-program speedup.

A 100 s baseline has 20 s serial and 80 s perfectly parallel work with no overhead. Four processors give T=20+80/4=40 s and speedup 2.5. Generally, $T(P) = 100[s + (1 - s)/P]$, so speedup is at most $1/s$, before synchronization and data-movement costs. Operation-count fractions cannot replace baseline-time fractions. Hardware does not automatically discover the parallel work.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multiprogramming overlaps CPU and I/O

Assume one CPU, one I/O device, free switching, A=CPU 2→I/O6→CPU 2, B=CPU 4, both ready at t=0.

Policy	CPU intervals	I/O	All complete	CPU utilization
Finish A before B	A0–2, idle2–8, A8–10, B10–14	A2–8	14	8/14=4/7
Run B while A blocks	A0–2, B2–6, idle6–8, A8–10	A2–8	10	8/10=0.8

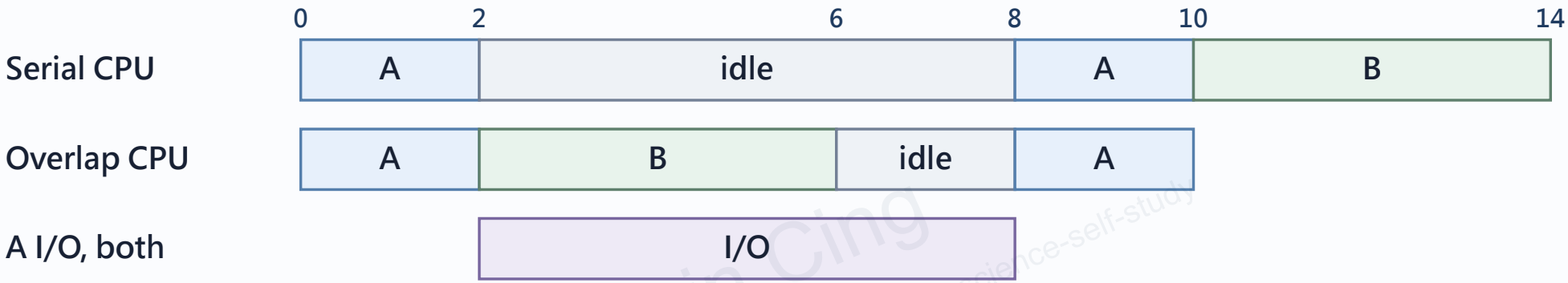
CPU work remains 8. One CPU does not execute both CPU bursts simultaneously. The saving of 4 is exactly the overlap of B with A's I/O. Without other ready work, or when all I/O waits on one bottleneck, this saving need not exist.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Multiprogramming overlaps CPU and I/O (continued)

Same workload: A=CPU 2→IO6→CPU 2, B=CPU 4

One CPU, independent I/O, free switches. Blue: A. Green: B. Gray: idle. Purple: I/O.



CPU work8 in both. Serial: 14, utilization4/7. Overlap: 10, utilization0.8. Saving4 is B/I/O overlap.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Time sharing requires recoverable control

Switching only on blocking cannot protect other jobs from an infinite CPU loop. Time sharing normally adds timer preemption and a ready-queue rule. For n continuously ready jobs, quantum q, switching cost c, and fixed RR, a job that just missed its turn waits at most $(n-1)q+nc$, conservatively including switch costs. The bound fails with unbounded n, unbounded nonpreemptible kernel execution, or indefinitely inserted higher-priority work.

Smaller q reduces rotation delays but raises $c/(q+c)$. With q=4 ms and c=0.1 ms, this CPU-only model gives 0.1/4.1≈2.44%. This is a specified model, not measured universal overhead.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Privilege and execution context

Protected privilege state prevents ordinary code changing the timer, page-table root, or interrupt entry. A system call saves ISA-required return state and enters controlled kernel code. Validation of arguments and authority precedes service.

t	One thread's execution	Mode	Process switch required?
0	Prepare read arguments	User	No
1	Entry and validation	Kernel	No
2a	Cache hit, copy, return	Kernel→user	No
2b	Cache miss, wait for I/O	Blocked in kernel	Another ready thread may run

A mode switch changes privilege. A context switch saves and loads an execution context. Another thread in the same address space need not require an address-space switch.

Privilege and execution context (continued)

Cache-hit syscall: 2 mode transitions,0 thread switches



Blue: user. Green: kernel. Entry/return change mode; P and its AS stay the same.

No other event here. Blocking/preemption may schedule Q. Count each switch type separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A timer does not prove fairness

The OS sets a protected timer before handing control to untrusted user code. Allowing that code to disable or modify the timer defeats control recovery. Progress also requires serviceable interrupts, bounded nonpreemptible kernel sections, terminating handlers and scheduler actions, and selection without permanent starvation.

Interrupts can be delayed, batched, or prioritized. Setting q does not imply a precise switch at each q boundary. Kernel faults and malicious arguments remain possible. Mode enforcement is a foundation, not a proof of all kernel behavior.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Programs and process state

A program is passive bytes, perhaps in a file. A process has an address space, credentials, open references, and at least one thread. A thread has a PC, registers, stack, and execution context. Threads share some process resources, not one PC.

Event	Transition	Actor
Wait for I/O	Running→waiting	Current thread and kernel register waiting
I/O completion	Waiting→ready	Completion path makes it schedulable
Selection	Ready→running	Scheduler and dispatcher
Quantum expiration	Running→ready	Controlled preemption

Completion does not immediately make a thread running; it must obtain the CPU. Blocked time is not ready-queue waiting. Process management also includes creation, termination, synchronization, IPC, and cleanup.

A non-atomic increment loses updates

Initially counter=0. Each thread loads, increments, and stores.

Step	T1	T2	Counter
1	Load r1=0		0
2		Load r2=0	0
3	Store r1+1=1		1
4		Store r2+1=1	1

The result is 1 rather than 2. A single CPU can switch between these steps; multiple CPUs add interleavings. A suitable mutex or atomic RMW changes the contract to an indivisible update. `volatile` alone does not provide mutual exclusion. Chapter 5 supplies the ordering conditions.

Equal virtual addresses, different frames

Pages are 4096 B. P maps VPN 1→PFN 7 read/write; Q maps VPN 1→PFN 9 read-only. VA0x1234 has VPN 1 and offset0x234=564. P's PA is 0x7234, Q's 0x9234.

Operation	P	Q
Read VA0x1234	Frame 7+564	Frame 9+564
Write VA0x1234	Permission allows	Permission fault
Change page table to frame 7	Ordinary user cannot modify protected tables	Same restriction

Mapping existence, residency, and authorization are distinct. The OS may repair a not-present mapping; adding a page need not repair forbidden writing. Sharing requires explicit mappings to the same frame and defined permissions. Equal VAs do not create sharing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Frame allocation and reclamation

Allocated and free sets cannot both claim exclusive ownership of one frame. Legitimate sharing needs consistent references and permissions. Reclamation requires old mappings to be unusable, including TLB entries and outstanding I/O holders.

Eight RAM frames with kernel 2, P3, Q2 leave free 1. Releasing one unshared Q frame gives free 2. If P still references it, it cannot enter the free pool. Remaining reachability matters more than one process declaring release. Virtual memory need not keep all virtual pages resident, but RAM remains finite. Working sets, backing I/O, and commit limits constrain performance and allocation guarantees.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Define miss penalty before averaging

L1 lookup costs 1 cycle, hit probability is 0.9, and a miss adds 50 cycles. Without overlap or prefetch, mean cost is $1+0.1\times50=6$, equivalently $0.9\times1+0.1\times51$. If 50 denotes total miss latency, the mean is $0.9\times1+0.1\times50=5.9$. The inputs differ. A mean of 6 does not make every access 6 or specify tail latency.

Caching introduces copy coherence and invalidation. CPU coherence does not establish filesystem durability; data may remain volatile. Write completion, flush completion, and power-loss survival need separate definitions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Allocation and storage failures

Applications use file names and offsets. Filesystems map ranges to blocks, and block layers and drivers submit device requests. Allocation prevents duplicate exclusive owners; free-space indexes must agree with object pointers. A memory lock handles concurrency but cannot repair a crash that persisted half an update.

For 2^{20} blocks at one bit per block, bitmap capacity is 2^{20} bits=2^{17} B=128 KiB. This is logical capacity. An uncached query still reads its containing storage block, not one physical bit. Corruption, hardware failure, accidental deletion, and hostile access need different responses. RAID, backup, and protection do not replace one another.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Protection and security

Protection restricts known subjects' use of objects. Security also concerns authentication, intrusion, malware, and threats to the system. A simple internal/external split cannot classify every attack.

Question	Mechanism	Still insufficient
Who requests?	Authentication and trusted credentials	Identity does not grant authority
What object and operation?	ACL or capability and access check	UI visibility cannot enforce an API
Can messages be intercepted?	Threat-appropriate encryption and key management	Cannot repair compromised endpoints
Can mappings exceed authority?	MMU rights and protected tables	Kernel bugs may bypass policy

Confidentiality, integrity, and availability are separate. Denying every request may prevent disclosure while destroying availability.

Least privilege and authorization time

An image converter needs input-file read and specified-output write rights, not write authority over a whole home directory. Bounded authorized references limit damage better than unrestricted paths or root identity. An ordinary user still has authority through inherited descriptors, shared memory, and sockets.

Removing a name's permission need not invalidate existing handles. Distinguish reference-creation checks from later-operation checks. A complete decision identifies credentials, object identity, requested rights, policy, and check time. File names, displayed identities, and privilege icons alone are insufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kernel structures and their premises

Structure	Operations	Conditions
Linked list	O(1) insertion/deletion with required node or predecessor	Worst-case key search O(n), pointer/cache costs
Stack or FIFO queue	O(1) endpoints with suitable representation	FIFO is not priority order
Balanced search tree	Ordered lookup/ranges, O(log n)	Maintain balancing invariant
Hash table	Average O(1) in specified model	Worst-case collision O(n), resizing costs
Bitmap	Membership in finite integer domain	Space follows domain size; claims need atomicity

With distinct BST keys, all left-subtree keys are below the node and all right-subtree keys above. Immediate-child comparisons are insufficient. Balanced height is O(log n), not an exact log2 n level count for any tree.

Collisions, subtree bounds, and lifetime

For h(k)=k mod4, tail-insert 5,9,13 into bucket1 as [5,9,13]. Searching 13 takes three comparisons, and absent17 also takes three. n congruent keys give linear worst-case search. Constant average cost requires distribution/load premises or a stronger design.

A tree with root 10, left5, right20, and 12 as 5's right child satisfies 12>5 but violates root 10's left-subtree bound. Searching 12 follows right20 and then an empty left link. Full subtree ranges must hold.

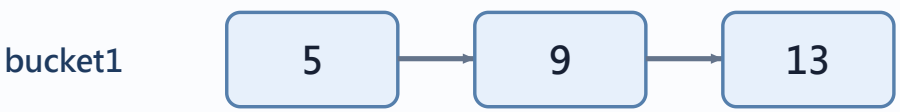
A reader may retain an old node after unlinking. Immediate free/reuse permits use-after-free. Locks, reference counting, or an RCU-style grace period have different reclamation contracts. Stale values do not authorize invalid memory.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Collisions, subtree bounds, and lifetime (continued)

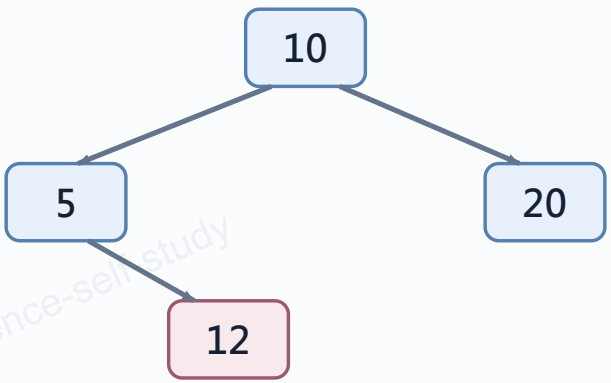
Hash collision cost and complete BST subtree bounds

h(k)=k mod4; chaining



Find 13: 3 comparisons. Reject 17: 3 comparisons.

n congruent keys: worst case O(n).



Red 12 is left of root 10 but12>10: subtree-bound violation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Computing environments

Environment	Boundary	Unsupported inference
Client/server	Request and service roles	A server cannot also be a client
Peer-to-peer	Nodes request and provide	No central discovery/authentication exists
Cluster/distributed	Independent nodes and network	Timeout proves remote failure
VM	Guest virtual hardware through VMM	Guest privilege equals host privilege
Conventional container	Views normally share host kernel	Each has an independent kernel
Embedded/real-time	Device or timing constraints	Every embedded system is hard real-time

IaaS, PaaS, and SaaS divide provision and management responsibilities. They are not CPU architectures. Virtualization can support them, but every cloud service is not one VM.

Deadlines are part of correctness

Assume one CPU, no blocking/overhead, A(C=1, T=D=4), B(C=2, T=D=6). Demand is 1/4+2/6=7/12. Utilization below one alone does not prove deadlines without a scheduler.

A feasible 12-unit EDF schedule is A0–1, B1–3, idle3–4, A4–5, idle5–6, B6–8, A8–9, idle9–12. A completes at1,5,9 before4,8,12; B at3,8 before6,12. It repeats because lcm=12 and no work remains at the boundary. Nonpreemptible sections, jitter, or I/O change the premises and require fresh blocking/response analysis. Low average latency does not replace a worst-case deadline check. Hard/soft real-time differ in consequences and permitted violations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Source visibility and open-source rights

The Open Source Definition requires more than readable source, including specified redistribution and derived-work rights. Source-available, freeware, public domain, and open source are different. Commercial charging need not conflict with open source.

A distribution combines a Linux kernel with tools, libraries, services, and packages. BSD is another family and licensing tradition. One open component does not give all files one license. Comparing implementations requires fixed versions, configuration, and dependencies. Linux 3.2 and Windows 7 retain historical contracts here; modern supplementary interfaces are identified separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Match evidence to the question

Question	Evidence	Insufficient substitute
Which API outcomes are allowed?	Versioned interface/standard	One successful run
How does this version implement it?	Matching source/configuration/architecture	Same function name in another version
What is observed latency?	Workload time/resource breakdown	Operation proportions treated as time
Is an update durable?	Storage/flush contract and failure order	Write returned success

Source access permits inspection, not a proof of no races, optimal speed, or security. Preserve premises, error outcomes, and general invariants. One example does not establish all inputs.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

One read connects the course

P reads4096 B through an authorized fd. Controlled entry validates the descriptor/OFD and buffer. A cache miss creates I/O and blocks P; the scheduler selects Q. DMA completion makes P ready. P later obtains the CPU, observes valid data, and copies/returns actual bytes.

Part	State	Chapters
Call/protection	Mode, credentials, requested right	2,14,15
Process/thread	Ready/waiting/running, saved registers	3–7
Mapping/data	VA/PTE/frame, cache, buffer lifetime	8,9,13
Name/storage	OFD offset, inode, block, completion/durability	10–12

These states do not complete simultaneously. Waiting→ready is not immediately running. Completing one request does not flush unrelated files.

Quantities that must remain distinct

Mode transitions, context switches, and address-space switches are separate counts. One thread's cache-hit system call may require only the first. CPU busy time, request latency, throughput, and deadlines use different denominators; less CPU work need not shorten a request. Names, object identities, and open references differ; renaming a path need not alter an fd. Average cost, worst-case cost, and general correctness differ, as collisions, timer delay, and races demonstrate.

Fix the version and abstraction, enumerate observable states/results, then use invariants and counterexamples to assess the mechanism.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 1. Open Source Initiative: Open Source Definition.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study